

EXPERIMENT NO. 02

Aim:

2. Perform bag-of-words approach (count occurrence, normalized count occurrence), TF-IDF on data. Create embeddings using Word2Vec..

Theory:

Bag of Words (BoW)

Bag of words is a Natural Language Processing technique of text modelling.

A problem with modeling text is that it is messy, and techniques like machine learning algorithms prefer well defined fixed-length inputs and outputs. Machine learning algorithms cannot work with raw text directly; the text must be converted into numbers. Specifically, vectors of numbers. This is called feature extraction or feature encoding.

The bag-of-words model is a way of representing text data when modeling text with machine learning algorithms. It is a popular and simple method of feature extraction from text data.

A bag-of-words is a representation of text that describes the occurrence of words within a document. It involves two things:

1. A vocabulary of known words.
2. A measure of the presence of known words.

It is called a “bag” of words, because any information about the order or structure of words in the document is discarded. The model is only concerned with whether known words occur in the document, not where in the document.

The most common kind of characteristic, or feature calculated from the Bag-of-words model is term frequency, which is essentially the number of times a term appears in the text. Term frequency is not necessarily the best representation for the text, but it still does find successful applications in areas like email filtering. Term frequency isn't the best representation of the text because common words such as "the", "a", "to" are almost always the terms with highest frequency in the text. This shows that having a high raw count does not necessarily indicate that the corresponding word is more important.

Advantages of BoW Approach

The most significant advantage of the bag-of-words model is its simplicity and ease of use. It can be used to create an initial draft model before proceeding to more sophisticated word embeddings.

Disadvantages of BoW Approach

- **Vocabulary:** The vocabulary requires careful design, most specifically in order to manage the size, which impacts the sparsity of the document representations.
- **Sparsity:** Sparse representations are harder to model both for computational reasons (space and time complexity) and also for information reasons, where the challenge is for the models to harness so little information in such a large representational space.
- **Meaning:** Discarding word order ignores the context, and in turn meaning of words in the document (semantics). Context and meaning can offer a lot to the model, that if modeled could tell the difference between the same words differently arranged ("this is interesting" vs "is this interesting"), synonyms ("old bike" vs "used bike"), and much more.

Bag-of-words example

Let's assume we have three sentences in our vocabulary.

Sentence 1: Data science is fun and interesting

Sentence 2: Data science is fun

Sentence 3: science is interesting

The unique words in the sentences are : [data, science, is, fun, and, interesting]. Hence, the bag of words vectors for the above sentences will be

Sentence 1: [1, 1, 1, 1, 1, 1]

Sentence 2: [1, 1, 1, 1, 0, 0]

Sentence 3: [1, 1, 1, 1, 0, 0]

Bag of Words Algorithm Implementation

In [6]:

```
""" vectorize() function takes list of words in a sentence as input
    and returns a vector of size of filtered_vocab. It puts 0 if the
    word is not present in tokens and count of token if present."""
def vectorize(tokens):
    vector=[]
    for w in filtered_vocab:
        vector.append(tokens.count(w))
    return vector
"""unique() functions returns a list in which the order remains
    same and no item repeats. Using the set() function does not
    preserve the original ordering, so i didnt use that instead"""
def unique(sequence):
```

```

seen = set()
return [x for x in sequence if not (x in seen or seen.add(x))]
#create a list of stopwords. You can import stopwords from nltk too
stopwords=["to","was","a"]
#list of special characters. You can use regular expressions too
special_char=["",".",",",";","'",'"',"?"]

string1 = "Data science is fun and interesting"
string2 = "Data science is fun"
string3 = "science is interesting"

#converting strings to lower case
string1=string1.lower()
string2=string2.lower()
string3=string3.lower()

#split the sentences into tokens
tokens1=string1.split()
tokens2=string2.split()
tokens3=string3.split()

print(tokens1)
print(tokens2)
print(tokens3)
#create a vocabulary list
vocab=unique(tokens1+tokens2+tokens3)
print(vocab)
#filter the vocabulary list
filtered_vocab=[]
for w in vocab:
    if w not in stopwords and w not in special_char:
        filtered_vocab.append(w)
print("Final filtered vocabulary: ", filtered_vocab)
#convert sentences into vectors
vector1=vectorize(tokens1)
print("Sentence 1 vector :",vector1)
vector2=vectorize(tokens2)
print("Sentence 2 vector :",vector2)
vector3=vectorize(tokens3)
print("Sentence 3 vector :",vector2)

['data', 'science', 'is', 'fun', 'and', 'interesting']
['data', 'science', 'is', 'fun']
['science', 'is', 'interesting']
['data', 'science', 'is', 'fun', 'and', 'interesting']
Final filtered vocabulary: ['data', 'science', 'is', 'fun', 'and', 'interesting']
Sentence 1 vector : [1, 1, 1, 1, 1, 1]
Sentence 2 vector : [1, 1, 1, 1, 0, 0]

```

Sentence 3 vector : [1, 1, 1, 1, 0, 0]

Creating Bag of Words using sklearn library

In [10]:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

string1 = "Data science is fun and interesting"
string2 = "Data science is fun"
string3 = "science is interesting"

doc = string1+string2+string3

CountVec = CountVectorizer(ngram_range=(1,1))
#transform
Count_data = CountVec.fit_transform([string1,string2,string3])

#create dataframe
cv_dataframe=pd.DataFrame(Count_data.toarray(),columns=CountVec.get_feature_names()
)
print(cv_dataframe)
```

	and	data	fun	interesting	is	science
0	1	1	1	1	1	1
1	0	1	1	0	1	1
2	0	0	0	1	1	1

/usr/local/lib/python3.8/dist-packages/sklearn/utils/deprecation.py:87: FutureWarning: Function get_feature_names is deprecated; get_feature_names_out is deprecated in 1.0 and will be removed in 1.2. Please use get_feature_names_out instead.
warnings.warn(msg, category=FutureWarning)

Note that the CountVectorizer sorts the vocabulary alphabetically before generating vectors.

Count Occurrence

In [14]:

```
count_vec = CountVectorizer()
count_occurs = count_vec.fit_transform([doc])
count_occur_df = pd.DataFrame((count, word) for word, count in
zip(count_occurs.toarray().tolist()[0], count_vec.get_feature_names_out()))
count_occur_df.columns = ['Word', 'Count']
count_occur_df.sort_values('Count', ascending=False, inplace=True)
count_occur_df.head()
```

Out[14]:

	Word	Count
6	is	3
7	science	2
0	and	1
1	data	1
2	fun	1

Normalized Count Occurrence

In [13]:

```

from sklearn.feature_extraction.text import TfidfVectorizer
norm_count_vec = TfidfVectorizer(use_idf=False, norm='l2')
norm_count_occurs = norm_count_vec.fit_transform([doc])
norm_count_occur_df = pd.DataFrame((count, word) for word, count in zip(
    norm_count_occurs.toarray().tolist()[0], norm_count_vec.get_feature_names_out()))
norm_count_occur_df.columns = ['Word', 'Count']
norm_count_occur_df.sort_values('Count', ascending=False, inplace=True)
norm_count_occur_df.head()

```

Out[13]:

	Word	Count
6	is	0.688247
7	science	0.458831
0	and	0.229416
1	data	0.229416
2	fun	0.229416

Conclusion: